S&DS 265 / 565

Introductory Machine Learning

Neural Networks (continued)

March 26

Yale

Version of 2026/03/25 at 10:47

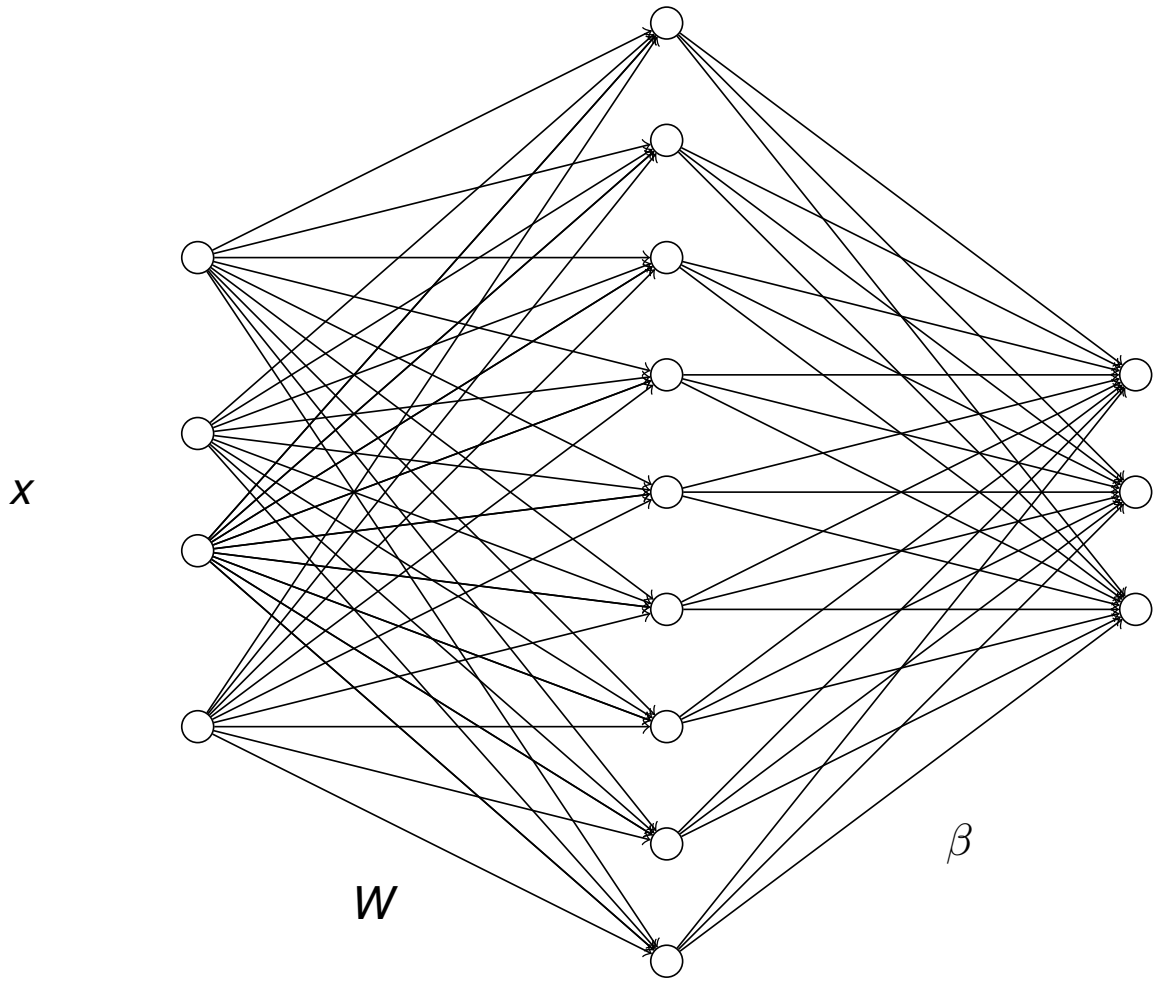
Reminders

- Assn 3 due tomorrow
- Assn 4 posted tomorrow
- Quiz 4 next week
- More volunteers for the iML societal issues panel? (sds265@yale)

Today

- Recap: Basic architecture of feedforward neural nets
- Backpropagation — high level, and sample calculations
- “np-complete” implementation
- Tensorflow Playground to build understanding, intuition
- Direct `numpy` implementation for classification

MLP structure



Nonlinearities

Add nonlinearity

$$h = \varphi(Wx + b)$$

fixed and *applied component-wise*

Typically the last layer is just linear (for both classification and regression):

$$f = \beta^T h + \beta_0$$

Nonlinearities

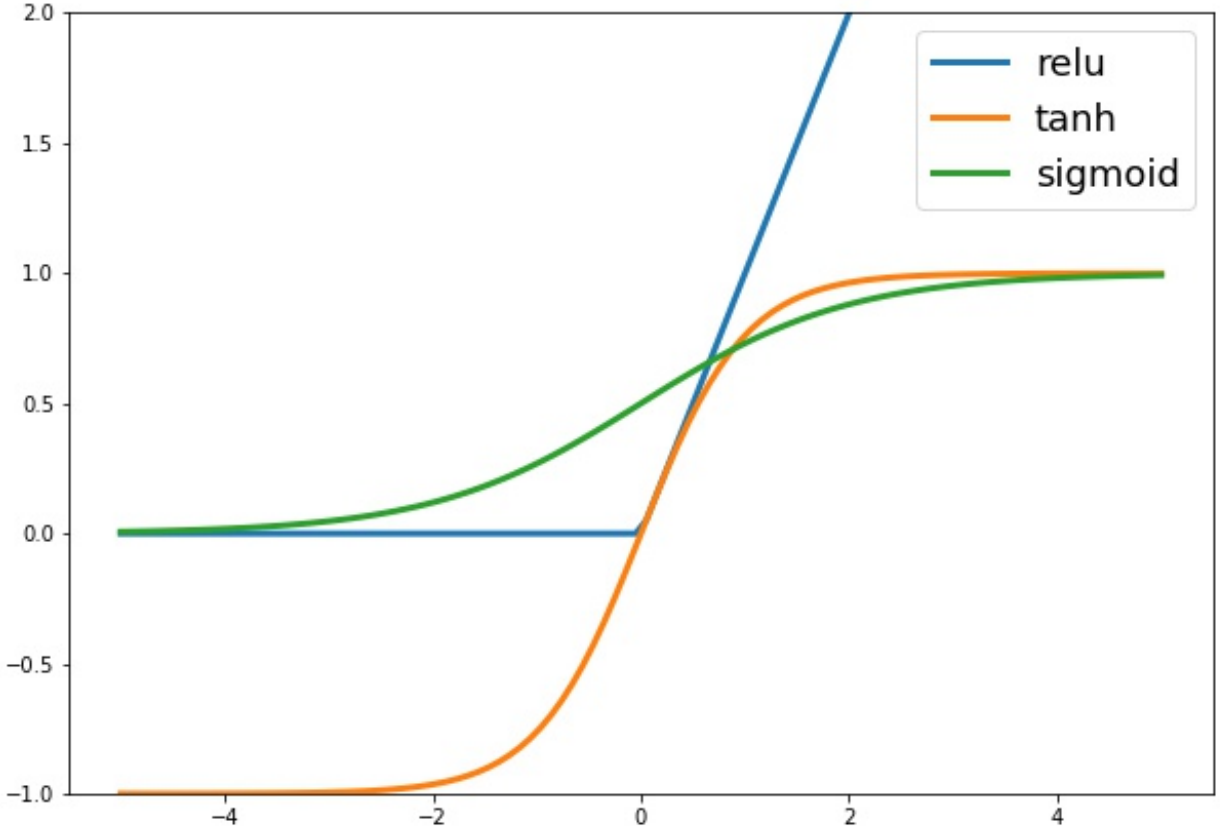
Commonly used nonlinearities:

$$\varphi(u) = \tanh(u) = \frac{e^{u} - e^{-u}}{e^{u} + e^{-u}}$$

$$\varphi(u) = \text{sigmoid}(u) = \frac{1}{1 + e^{-u}}$$

$$\varphi(u) = \text{relu}(u) = \max(u, 0)$$

Nonlinearities



Gelu

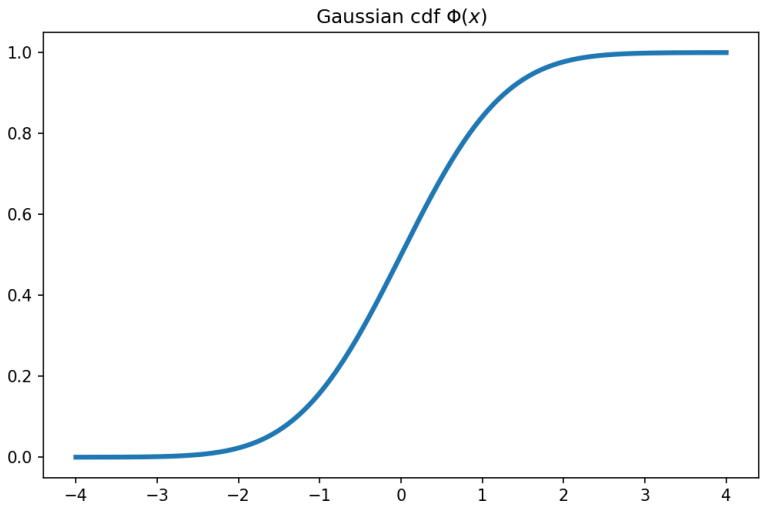
A recent “invention” is the Gaussian error linear unit (Gelu):

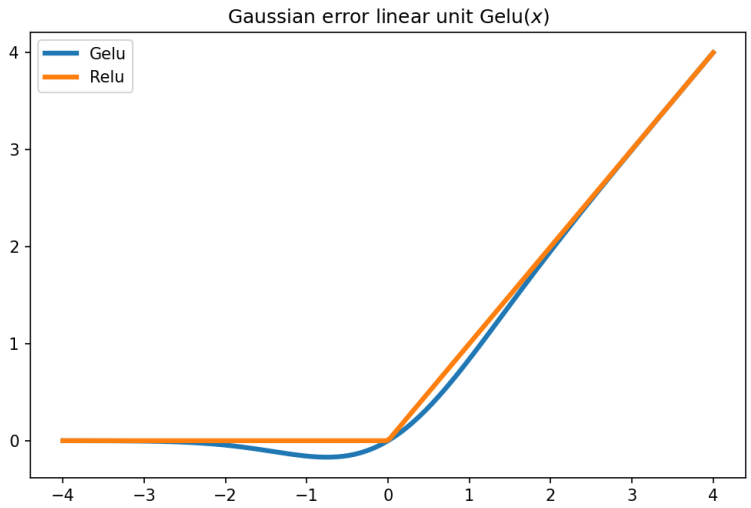
$$\varphi(u) = u \cdot \Phi(u)$$

where Φ is the Gaussian cumulative distribution function (aka the error function)

“Gaussian Error Linear Units (GELUs),” D. Hendrycks and K. Gimpel, arXiv:1606.08415

Gelu





"Gaussian Error Linear Units (GELUs)," D. Hendrycks and K. Gimpel, arXiv:1606.08415

Nonlinearities

So, a neural network is nothing more than a parametric regression model with a restricted type of nonlinearity

The “Cat neuron”

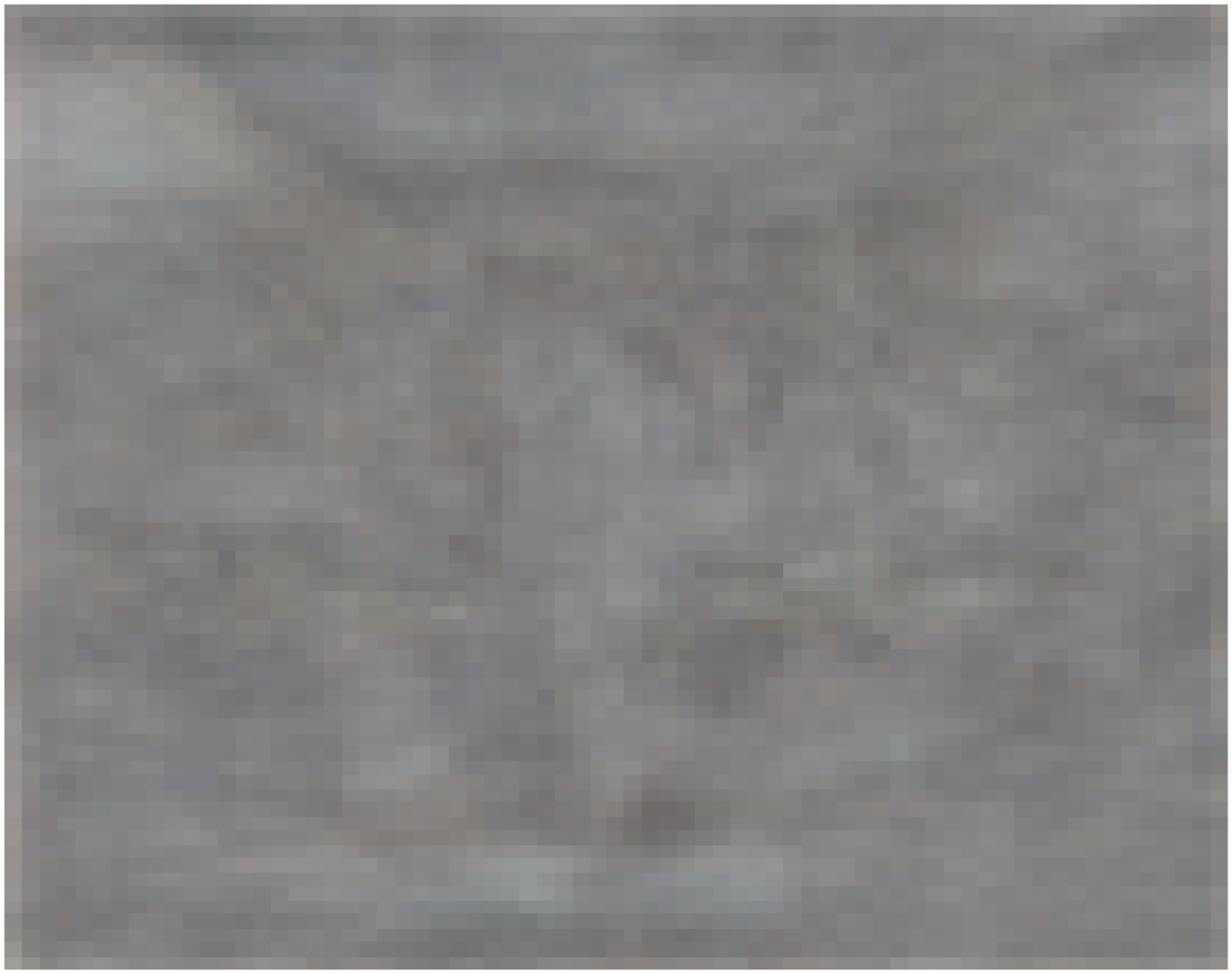
An Ahah! moment in AI history came in 2012

- Google Brain researchers (led by Andrew Ng and Jeff Dean) trained a convolutional neural net in an unsupervised fashion
- Data: 10 million *unlabeled* 200 $\times$ 200 pixel images from YouTube videos
- Trained for three days on 1,000 computers
- The network had a specific neuron that activated only for cat faces

---

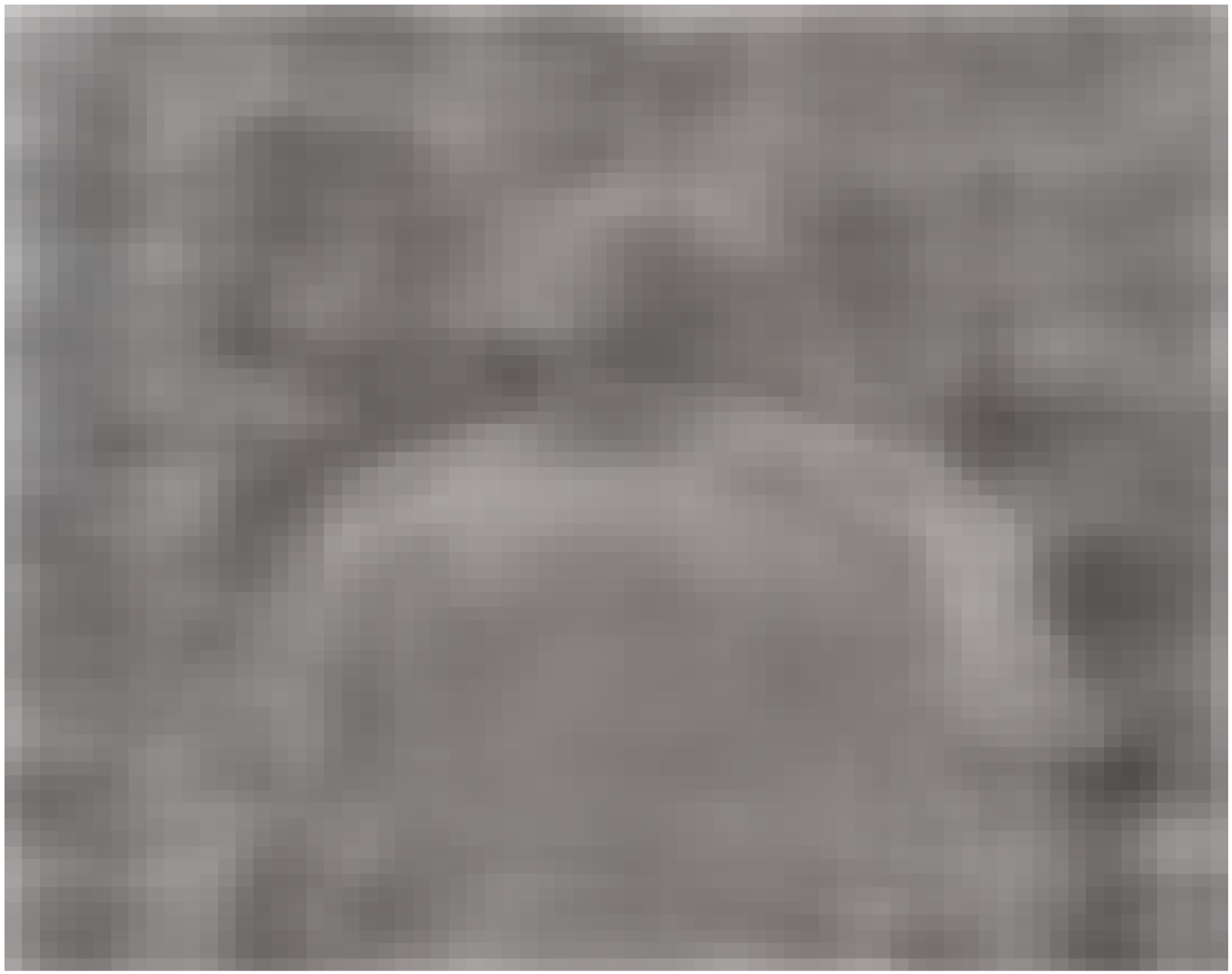
Le et al., “Building High-level Features Using Large Scale Unsupervised Learning,” 2012

The “Cat neuron”



Le et al., “Building High-level Features Using Large Scale Unsupervised Learning,” 2012

Body neuron



Le et al., “Building High-level Features Using Large Scale Unsupervised Learning,” 2012

Sanity check

Let's work through a toy example to be sure we understand the "mechanics" of the neural net

2-dimensional inputs $x = (x_1, x_2)^T$ and a binary classification

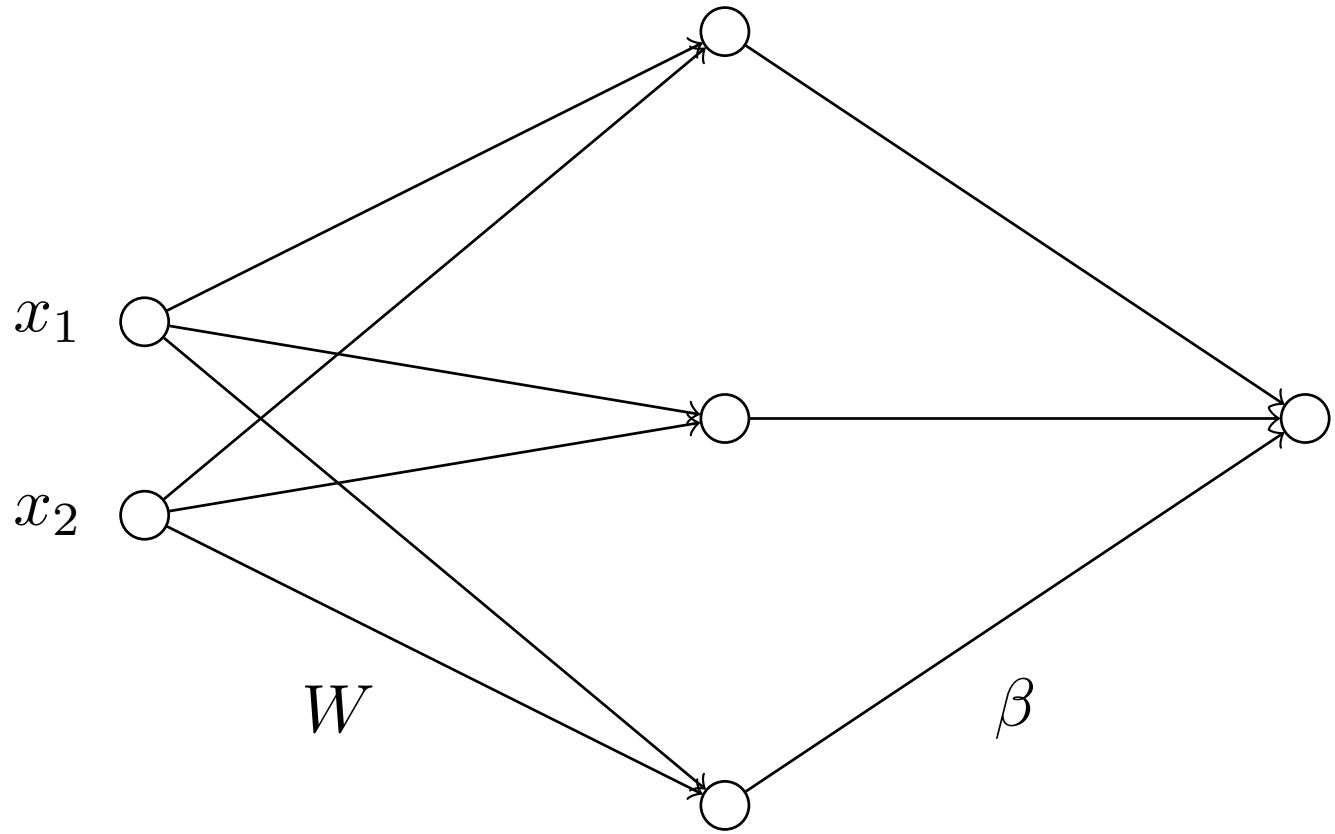
Sanity check

We have 2-layer neural net with three hidden neurons

$$h(x) = \varphi\left(Wx + b\right)$$
$$\beta^T h(x) = \log\left(\frac{p(Y=1 \mid x)}{p(Y=0 \mid x)}\right)$$

with $\varphi(u) = \mathit{relu}(u) = \max(u, 0)$

Sanity check



Sanity check

Suppose that the parameters (weights) of the network are

$$b = (1, 0, -1)^T$$

$$W = \begin{pmatrix} 1 & -1 \\ -2 & 1 \\ 0 & 1 \end{pmatrix}$$

$$\beta = (1, 2, -1)^T$$

What is the prediction of the network if $x = (1, -1)^T$ or $x = (-1, 1)^T$?

Sanity check

Suppose that the parameters (weights) of the network are

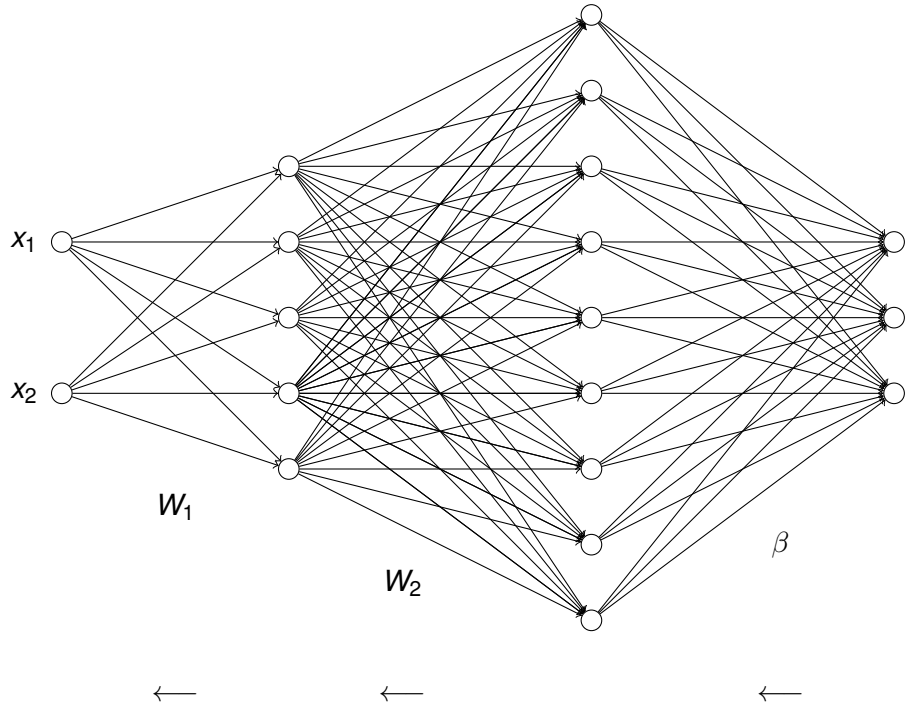
$$b = (1, 0, -1)^T$$
$$W = \begin{pmatrix} 1 & -1 \\ -2 & 1 \\ 0 & 1 \end{pmatrix}$$
$$\beta = (1, 2, -1)^T$$

How do the predictions change if $\varphi = \tanh$?

Training

- The parameters are trained by stochastic gradient descent.
- To calculate derivatives we just use the chain rule, working our way backwards from the last layer to the first.

High level idea



Start at last layer, send error information back to previous layers

Backprop calcs

We'll go through some of the backpropagation calculations. It's not essential that you can repeat all of these (but try!)

Details notes are posted to our course web page, under "Notes on backpropagation"

The basic rule

Suppose we have a matrix product $A = BC$, then

$$\frac{\partial \mathcal{L}}{\partial B} = \frac{\partial \mathcal{L}}{\partial A} C^T$$

$$\frac{\partial \mathcal{L}}{\partial C} = B^T \frac{\partial \mathcal{L}}{\partial A}$$

Check that the dimensions match up! The shape of $\frac{\partial \mathcal{L}}{\partial B}$ needs to be the same as that of B.

Example

So if $f = Wx + b$ then

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial f} \, x^T$$
$$= (f - y) \, x^T$$

Example

So if $f = Wx + b$ then

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial f}$$
$$= (f - y)$$

Two layers

Now add a layer:

$$f = W_2 h + b_2$$
$$h = W_1 x + b_1$$

Then we have

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial f} \, h^T$$
$$= (f - y) \, h^T$$

$$\frac{\partial \mathcal{L}}{\partial h} = W_2^T \, \frac{\partial \mathcal{L}}{\partial f}$$
$$= W_2^T \, (f - y)$$

Two layers

Now send this back (backpropagate) to the first layer:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_1} &= \frac{\partial \mathcal{L}}{\partial h}\, x^T \\ &= W_2^T\, \frac{\partial \mathcal{L}}{\partial f}\, x^T \\ &= W_2^T\, (f - y)\, x^T\end{aligned}$$

Adding a nonlinearity

Remember, this just gives a linear model! Need a nonlinearity:

$$h = \varphi(W_1 x + b_1)$$

$$f = W_1 h + b_2$$

Adding a nonlinearity

If $\varphi(u) = \mathit{relu}(u) = \max(u, 0)$ then this just becomes

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_1} &= \mathbb{1}(h > 0)\ \frac{\partial \mathcal{L}}{\partial h}\ x^T \\ &= \mathbb{1}(h > 0)\ W_2^T\ \frac{\partial \mathcal{L}}{\partial f}\ x^T \\ &= \mathbb{1}(h > 0)\ W_2^T\ (f - y)\ x^T\end{aligned}$$

where

$$\mathbb{1}(u) = \begin{cases} 1 & u > 0 \\ 0 & \text{otherwise} \end{cases}$$

See notes on backpropagation for details


You're not required to know the details of backprop!

np-Complete demo

Minimal neural network implementation

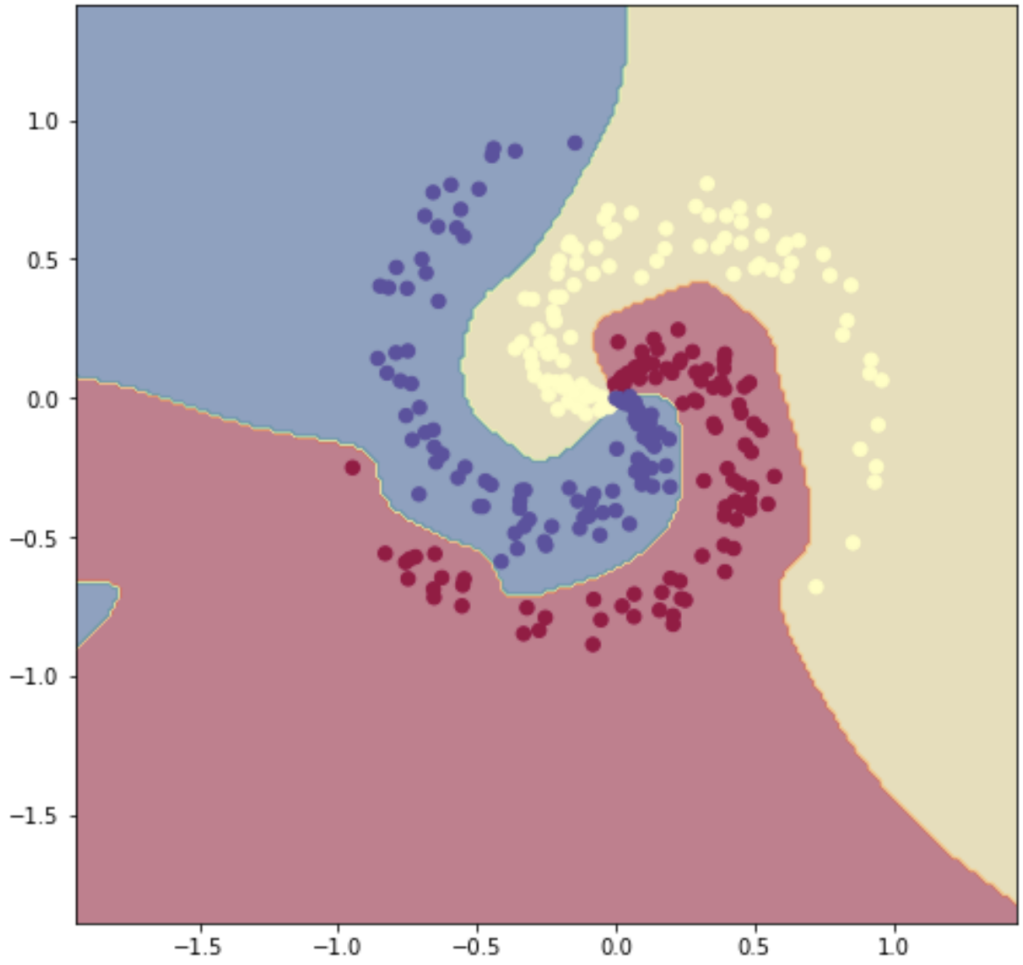
This is a "bare bones" implementation of a 2-layer neural network for classification, using rectified linear units as activation functions. The code is from Andrej Karpathy; please see this page for an annotated description of the code.

In assignment 7 you will extend this and experiment with some different settings of the parameters.

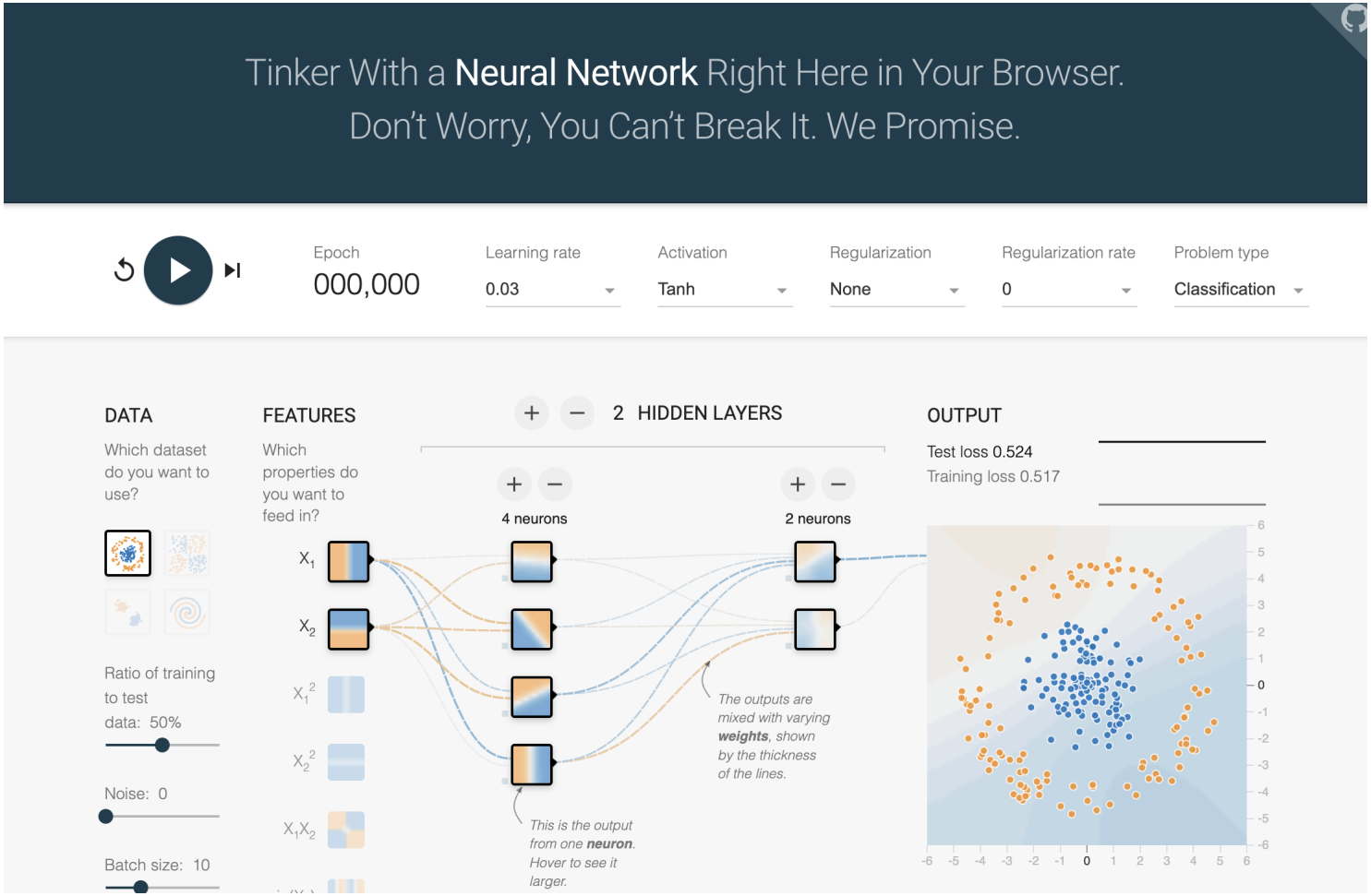
```
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
plt.rcParams['axes.facecolor'] = 'lightgray'
```

np-Complete demo



Brilliant interactive demo



https://playground.tensorflow.org/

Summary

- In neural networks, “features” are linear operations followed by an activation function
- Based on a crude analogy with neurons in biological brains
- Neurons are deterministic functions of input; not latent variables
- A special type of (parametric) nonlinear regression model
- Trained using stochastic gradient descent
- Backpropagation is “just” the chain rule from calculus
- Applied iteratively from the last layer backwards